

LAB NOTES · ISSUE 02 OF 04

Lab: a self-service deployment platform.

Request a service, get a validated, templated, GitOps-deployed app.

02

AR

Akhilesh Ranjan Singh

Senior DevSecOps Engineer · github.com/iarsingh

Swipe →

Request to running service

- 1 Service request
- 2 Validate or refuse
- 3 Render from template
- 4 Policy check
- 5 Terraform module + Argo CD
- 6 Service with /healthz and /readyz

Refusals come back with a reason. The platform never silently fixes a request.

Components

01 platform-api

FastAPI: service requests, status, metrics.

02 templates/python-service

The golden-path service skeleton.

03 policies/check_policy.py

Checks rendered manifests before deploy.

04 terraform/modules/service

Per-service infrastructure as a module.



Run it locally

BASH

```
./scripts/kind-up.sh
./scripts/install-argocd.sh
./demo/run.sh

# then break something on purpose
./scripts/simulate-failure.sh
```



What it demonstrates

01 Golden path

Every new service starts compliant.

02 Refusal over silent fixes

Developers learn the rules from the error.

03 Failure drills

A script to simulate breakage and practise recovery.



HONEST LABEL

A lab, not a customer platform.

It exists to practise the design. Scale, multi-tenancy and SSO are out of scope.



KEY TAKEAWAY

Self-service works when validation and refusal are first-class.

- Validate, refuse, render, deploy
- Templates carry the guardrails
- Practise failure on purpose

YOUR TURN

What would you refuse in a self-service platform request?

FOLLOW AKHILESH RANJAN SINGH

Next in the series: #03 Lab: MLOps on Kubernetes with GitOps

Save · Repost

